1. Introduction

This document explains the feature_engineering.py pipeline used to transform raw WiFi statistics into a structured time-series dataset for machine learning-based anomaly detection.

2. Nature of Raw Data

The input consists of cumulative counters collected from WiFi devices at fixed intervals (e.g., every 10 seconds). These counters continuously increase and do not directly represent real-time behavior.

3. Time-Series Formation

Data is sorted by client_mac and timestamp. For each device, the difference between consecutive timestamps is computed. This establishes a temporal sequence, where each row depends on previous observations.

4. Time Delta Computation

The time difference between consecutive samples (time_delta) is calculated. This value is critical for converting cumulative counters into rates per second.

5. Counter to Rate Conversion

Cumulative counters such as tx_data_packets are converted into rates using differencing: rate = (current value - previous value) / time_delta. This transforms static counters into dynamic time-series signals.

6. Activity Detection (is_active)

An is_active flag is introduced. It is set to 1 if either transmission or reception occurred during the interval, and 0 otherwise. This helps separate meaningful behavior from idle periods.

7. Reliability Metrics

Metrics such as tx_success_rate, retry_rate, tx_failed_rate, and rx_error_rate are computed. These represent transmission reliability and help identify degraded network conditions.

8. Throughput Features

Transmission and reception rates are converted to Mbps using available rate metrics. These features represent the efficiency and capacity of the link.

9. PHY Layer Features (MCS)

MCS (Modulation Coding Scheme) values represent link quality. The script computes low, mid, and high MCS ratios, a weighted MCS score, and entropy. These capture signal strength and stability over time.

10. Security Metrics

Security-related counters such as MIC errors, PN errors, and decryption errors are aggregated into a single feature. These indicate protocol-level anomalies.

11. Data Cleaning

Invalid values such as NaN or infinity are handled using replacement and filling strategies. This ensures compatibility with machine learning algorithms.

12. Final Dataset

The final dataset consists of selected features for each device at each timestamp. Each row represents the state of a device at a specific time interval.

13. Time-Series Interpretation

Each row is not independent. Instead, it represents a point in time in a sequence of observations. The dataset captures temporal evolution of network behavior.

14. Importance for Anomaly Detection

By converting raw logs into time-series features, the model can learn patterns of normal behavior. Deviations from these patterns are identified as anomalies.